

Practical 1

Aim: Breadth First Search & Iterative Depth First Search in Python.

- A) Implement the Breadth First Search algorithm to solve a given problem.
- B) Implement the Iterative Depth First Search algorithm to solve the same problem.
- C) Compare the performance and efficiency of both algorithms.

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Artificial Intelligence

CODE A:

```
S074 Ai Pract 1.py - C:/Users/PRIYANKA/Downloads/S074 Ai Pract 1.py (3.13.1)
File Edit Format Run Options Window Help
from collections import deque
#A) Implement the Breadth First Search algorithm to solve a given problem.
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F', 'G'],
    'D': ['B'],
    'E': ['B'],
    'F': ['C'],
    'G': ['C']
}

def bfs(start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        vertex = queue.popleft()
        print(vertex, end=" ")

        for neighbour in graph[vertex]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

print("BFS Traversal:")
bfs('A')
```

OUTPUT:

```
IDLE Shell 3.13.1
File Edit Shell Debug Options Window Help
Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/PRIYANKA/Downloads/S074 Ai Pract 1.py =====
BFS Traversal:
A B C D E F G
>>>
```

CODE B:

Adarsh Shukla
T117

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT NAME: Artificial Intelligence

```
*S074 Ai Pract 1.py - C:/Users/PRIYANKA/Downloads/S074 Ai Pract 1.py (3.13.1)*
File Edit Format Run Options Window Help
from collections import deque

#B) Implement the Iterative Depth First Search algorithm to solve the same problem.
graph = {
    1: [2, 3],
    2: [1, 4, 5],
    3: [1, 6],
    4: [2],
    5: [2, 7],
    6: [3],
    7: [5]
}

def iterative_dfs(start):
    visited = set()
    stack = [start]

    while stack:
        node = stack.pop()

        if node not in visited:
            visited.add(node)
            print(node, end=" ")

            for neighbour in reversed(graph[node]):
                if neighbour not in visited:
                    stack.append(neighbour)

print("DFS Traversal:")
iterative_dfs(1)
```

OUTPUT:

```
IDLE Shell 3.13.1
File Edit Shell Debug Options Window Help
Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/PRIYANKA/Downloads/S074 Ai Pract 1.py =====
DFS Traversal:
1 2 4 5 7 3 6
>>>
```

C: Comparison of BFS and DFS

BFS and Iterative DFS both have a time complexity of $O(V + E)$. BFS uses a queue and is better for finding the shortest path, while Iterative DFS uses a stack and usually requires less memory. Both algorithms efficiently traverse the graph, but they follow different traversal methods.